SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: **Theory of Computation** **COURSE CODE:** **CSA13 02**

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:50**
Duration : 1 Hour

Application Based Question

Code of Conduct:

I *M Poojitha Reddy - 192472352* ***(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.***
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
M Poojitha Reddy



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1.In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable.Design a **Deterministic Finite Automaton (DFA)** over the alphabet Σ = {0,1} to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2.In web applications, validating user input is essential to ensure correctness, security, and performance.Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications**.**

3.In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role.Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

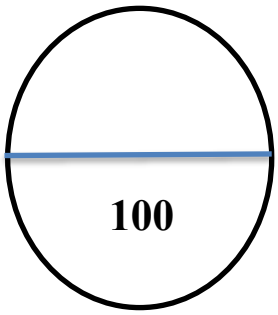
4.In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language L= { $a^n b^n$ | n ≥ 0 } cannot be recognized by any finite automaton.Further, analyze the implications of this limitation in the design of modern compilers.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules.Explain how the **closure properties of regular languages** enable the combination of multiple detection rules.Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

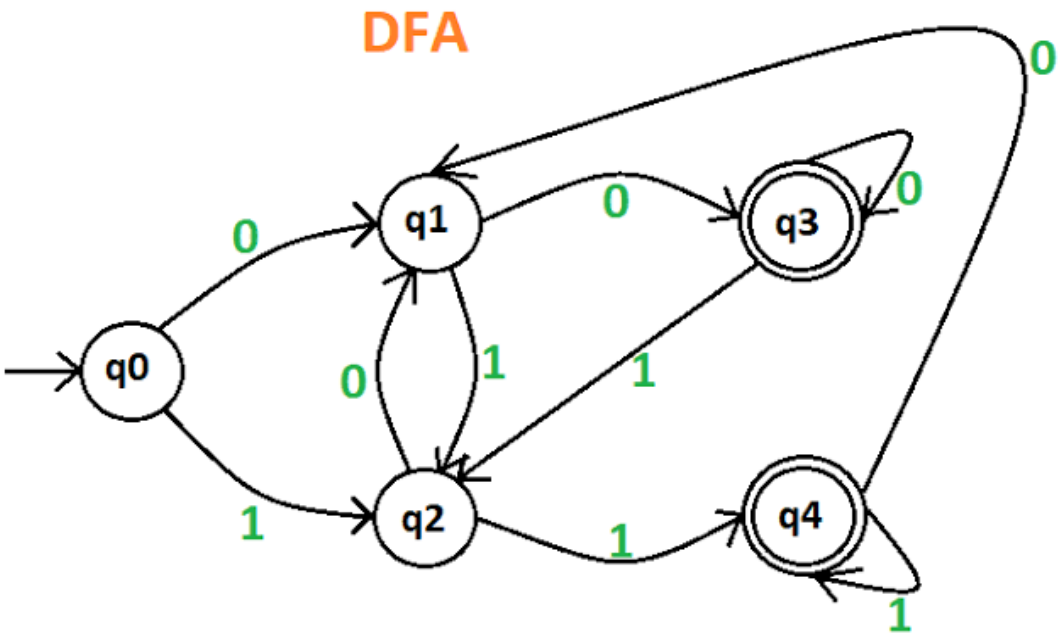


1. DFA for Passwords Ending with "01" and Its Minimization

Introduction

A **Deterministic Finite Automaton (DFA)** is a finite-state machine in which, for every state and input symbol, exactly one transition is defined. We need to design a DFA over **Σ = {0,1}** that accepts all strings ending with the pattern **"01"**.

DFA Design



We use three states:

- **q_0** – Initial state; no useful suffix of "01" has been detected.
- **q_1** – The last symbol read is 0.
- **q_2** – The string ends with 01; accepting state.

Transition Table

Current State	Input 0	Input 1
→ q_0	q_1	q_0
q_1	q_1	q_2
*q_2	q_1	q_0

For example:

- 01 → Accepted
- 101 → Accepted
- 0001 → Accepted
- 10 → Rejected
- 011 → Rejected

Minimization

The three states are **pairwise distinguishable**:

- q_2 is accepting while q_0 and q_1 are non-accepting.
- q_0 and q_1 are distinguishable because input 1 takes q_0 to q_0, whereas it takes q_1 to q_2.

Therefore, no two states can be merged.

Minimum number of states = 3.

Efficiency in Hardware Implementation

The minimized DFA improves system efficiency because:

1. It uses the **minimum number of states** required to recognize the pattern.
2. Fewer states require less memory for storing the state-transition logic.
3. Each input bit requires only **one deterministic transition**.
4. Hardware implementation can use simple flip-flops and combinational logic.
5. It provides predictable **O(n)** validation time for a password of length n.
6. It is suitable for high-speed applications such as ATMs, authentication modules, and embedded security devices.

Conclusion

The DFA with three states is already minimal and efficiently recognizes all binary strings ending in **"01"**. Its small state space, deterministic operation, and linear-time processing make it suitable for real-time hardware-based password validation.

2. Regular Expression for the Pattern “aaaabb”

Introduction

A **regular expression** is a formal notation used to describe patterns in strings. For the required language containing exactly the string **aaaabb**, the regular expression can directly specify the sequence of symbols.

Regular Expression

aaaabb

This expression accepts **only one string**:

aaaabb

Examples

Input	Result
aaaabb	Accepted
aaaab	Rejected
aaaaabb	Rejected
aaabb	Rejected
aaaabbba	Rejected

Equivalent Formal Language

The language is:

\[L = \{aaaabb\} \]

A regular expression can represent this language because a fixed string is a regular language.

Use in Web Applications

Regular expressions are widely used for input validation in web applications. They can validate:

- Usernames
- Password formats
- Email addresses

- Phone numbers
- PIN codes
- Identification numbers
- Form fields

For example, a web application can compare user input against a predefined regular expression before accepting the data.

Advantages

1. **Fast validation:** Regular expressions can process simple patterns efficiently.
2. **Compact representation:** Complex patterns can be represented concisely.
3. **Easy integration:** Most programming languages provide regular-expression libraries.
4. **Improved security:** Input validation can prevent malformed or unexpected input.
5. **Reduced processing:** Invalid input can be rejected before further application processing.

Conclusion

The regular expression **aaaabb** precisely validates the required input pattern. Regular expressions and regular languages provide an efficient method for validating structured user input in modern web applications.

3. Conversion of Regular Expression a*b* into a Finite Automaton

Introduction

The regular expression

\[a^*b^* \]

represents all strings containing **zero or more as followed by zero or more bs**.

Examples of accepted strings are:

ε

a

aa

b

bb

aab

aaabb

Step 1: Understand the Pattern

The expression has two parts:

- a* → any number of as
- b* → any number of bs

Once the automaton starts reading b, it cannot return to reading a.

Step 2: Construct the DFA

Use three states:

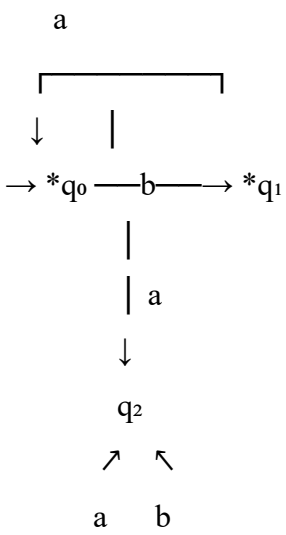
- **q_0** – Reading as; initial state.
- **q_1** – Reading bs.
- **q_2** – Dead state for an invalid a after bs.

Both q_0 and q_1 are accepting states because a* and b* can independently contain zero symbols.

Transition Table

State	a	b
→ *q_0	q_0	q_1
*q_1	q_2	q_1
q_2	q_2	q_2

A clearer representation is:



Why Regular Expressions and Finite Automata Are Equivalent

Regular expressions and finite automata are equivalent because they describe exactly the **regular languages**.

There are two important conversions:

1. **Regular Expression → Finite Automaton**
   - A regular expression can be converted into an NFA using standard construction techniques.
   - The NFA can then be converted into a DFA.
2. **Finite Automaton → Regular Expression**

- A finite automaton can be converted into an equivalent regular expression using methods such as state elimination.

Therefore:

$$\boxed{\text{Regular Expressions} \equiv \text{Finite Automata}}$$

Real-Time Detection Applications

This equivalence is useful in:

- Intrusion detection systems
- Spam filtering
- Network packet inspection
- Lexical analysis in compilers
- Log monitoring
- Malware signature detection
- Input validation

Finite automata process input **one symbol at a time**, making them suitable for high-speed and real-time pattern detection.

Conclusion

The regular expression a*b* can be represented by an equivalent finite automaton. Since both models recognize regular languages, they provide different but equivalent methods for efficient pattern recognition in real-time computing systems.

---

4. Pumping Lemma Proof for $L = \{a^n b^n \mid n \geq 0\}$

Introduction

The language

$$ L = \{a^n b^n \mid n \geq 0\} $$

contains strings having an equal number of as followed by an equal number of bs.

Examples:

ε

ab

aabb

aaabbb

aaaabbbb

We will prove using the **Pumping Lemma** that this language is **not regular**. Therefore, it cannot be recognized by any finite automaton.

Pumping Lemma

If a language L is regular, there exists a pumping length **p** such that every string $w \in L$, where $|w| \geq p$, can be divided as:

$$ w = xyz $$

such that:

1. $|xy| \leq p$
2. $|y| > 0$
3. $xy^iz \in L$ for every $i \geq 0$

Proof by Contradiction

Assume that L is regular.

Let **p** be the pumping length.

Choose the string:

$$ w = a^p b^p $$

Clearly:

$$ w \in L $$

Since $|xy| \leq p$, both x and y must occur entirely within the first group of as.

Therefore:

$$ y = a^k $$

where:

$$ k > 0 $$

Now pump y twice by taking $i=2$:

$$ xy^2z $$

This produces:

$$ a^{p+k}b^p $$

The number of as is now greater than the number of bs.

Therefore:

$$ a^{p+k}b^p \notin L $$

This contradicts the pumping lemma.

Hence, our assumption is false.

$$ \boxed{L=\{a^n b^n\mid n\geq0\}\text{ is not regular}} $$

Implication for Finite Automata

A finite automaton has only a **finite number of states** and cannot remember an arbitrarily large number of as to compare with the number of subsequent bs.

Therefore, no finite automaton can recognize this language.

Implications for Compiler Design

This limitation is important in compiler design:

1. **Lexical analysis** uses finite automata because tokens generally form regular languages.
2. **Syntax analysis** requires more powerful models such as context-free grammars and pushdown automata.
3. Nested structures such as balanced parentheses cannot be handled by finite automata alone.
4. A compiler therefore uses different models at different phases.

5. Finite automata are efficient for token recognition, while parsers handle more complex structural relationships.

Conclusion

The pumping lemma proves that \(L=\{a^n b^n\mid n\geq0\}\) is not regular. Hence, finite automata cannot recognize languages requiring unlimited counting or matching. This explains why modern compilers use finite automata for lexical analysis and more powerful parsing techniques for syntax analysis.

---

5. Closure Properties of Regular Languages

Introduction

Closure properties describe the operations under which a class of languages remains within the same class. Regular languages are closed under several important operations. This means that combining or modifying regular languages using these operations produces another **regular language**.

Important Closure Properties

If \(L_1\) and \(L_2\) are regular languages, then the following are also regular:

Operation	**Result**
Union	\(L_1 \cup L_2\)
Intersection	\(L_1 \cap L_2\)
Complement	\(\overline{L_1}\)
Difference	\(L_1-L_2\)
Concatenation	\(L_1L_2\)
Kleene Star	\(L_1^*\)
Reversal	\(L_1^R\)

Combining Detection Rules

Suppose an intrusion detection system has two rules:

$$L_1 = \text{strings containing “attack”} \quad L_2 = \text{strings containing “malware”}$$

If we want to detect inputs satisfying either rule:

$$L = L_1 \cup L_2$$

Since regular languages are closed under union:

$$L \text{ is regular}$$

Similarly, if an input must satisfy both conditions:

$$L = L_1 \cap L_2$$

the resulting language is also regular.

Why Does the Result Remain Regular?

Every regular language can be represented by a finite automaton.

If two languages have finite automata:

$$M_1 \text{ recognizes } L_1 \quad M_2 \text{ recognizes } L_2$$

we can construct another finite automaton that recognizes the combined language.

For example, for intersection, a **product automaton** can track the state of both machines simultaneously.

Therefore, the resulting automaton still contains only a **finite number of states**, so the resulting language remains regular.

Real-World Applications

Closure properties allow multiple pattern-matching rules to be combined efficiently in:

- **Intrusion Detection Systems:** Combine multiple attack signatures.
- **Spam Filtering:** Combine rules for suspicious words, links, and formatting.
- **Input Validation:** Combine username, password, and format requirements.
- **Compilers:** Combine lexical rules for different types of tokens.
- **Network Monitoring:** Combine multiple packet or protocol patterns.
- **Log Analysis:** Detect several types of abnormal events.

Advantages

1. Multiple detection rules can be combined systematically.

2. The resulting language remains regular.
3. The combined language can still be represented using finite automata.
4. Finite automata provide efficient and predictable pattern matching.
5. This makes regular-language-based systems suitable for real-time applications.

Conclusion

The closure properties of regular languages provide a mathematical foundation for combining multiple pattern-matching rules. Since operations such as union, intersection, concatenation, complement, and Kleene star preserve regularity, the resulting language can still be recognized by a finite automaton. This enables efficient implementation of complex detection systems in security, compilers, web applications, and real-time monitoring.